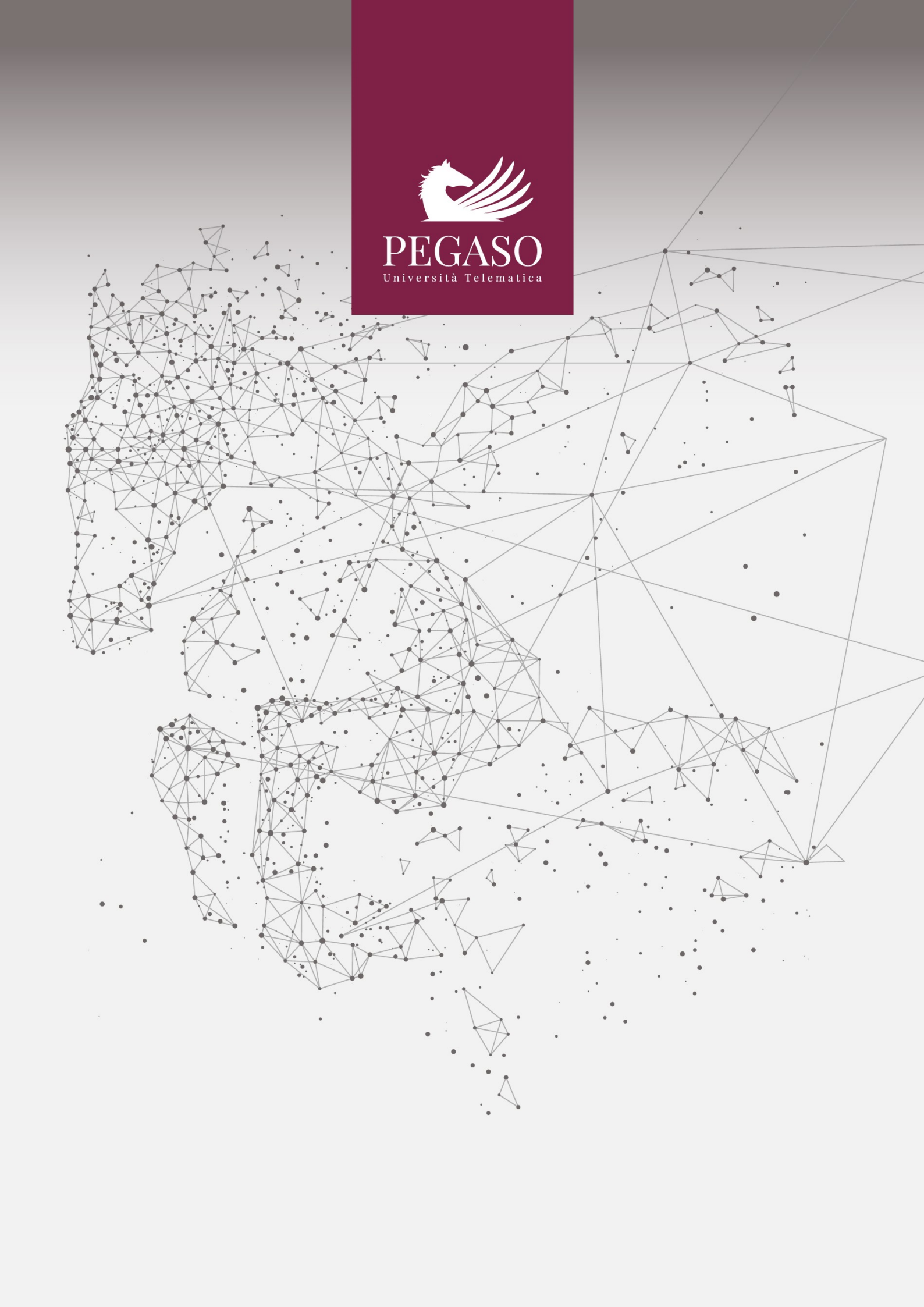

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. TIPI DI DATO NUMERICI....4
3. TIPI DI DATO ORARIO, DI DATA E STRINGA....7
4. IL COMANDO CREATE....11
BIBLIOGRAFIA....15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustrano due aspetti molto importanti del SQL. In particolare, si presentano i tipi di dato SQL e il comando CREATE del linguaggio DDL di SQL. La conoscenza dei tipi di dato esistenti in SQL raggruppabili in tre diverse classi (numerici, data/orario, stringa) è fondamentale per il corretto utilizzo e la corretta interazione con un DBMS basato su SQL. Il comando CREATE, invece, è fondamentale per la creazione di database, tabelle, indici e vincoli di integrità referenziale, attività molto utile per la creazione di database completi per l'archiviazione e la gestione dei dati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Tipi di dato numerici

Il linguaggio SQL (Structured Query Language) è un linguaggio di interrogazione strutturato. SQL consente di interagire con i dati presenti nelle basi di dati sia a livello di schema (struttura) – i campi che costituiscono la tabella – sia a livello di tupla (riga della tabella).

Per interazione in questo caso si intende ricercare, inserire, modificare, cancellare, operare con funzioni gestionali e amministrative sul DB.

In generale i comandi SQL si classificano in due gruppi fondamentali:

- DDL (Data Definition Language) - comandi per la definizione della struttura dei dati e l'amministrazione del database;
- DML (Data Manipulation Language) - comandi per l'interrogazione dei dati e la modifica dei dati.

In questa lezione ci focalizzeremo principalmente sui tipi di dato trattati da SQL e sul comando CREATE (DDL) per la definizione della struttura e dei vincoli nelle basi di dati. Prima di procedere con i comandi DDL veri e propri, è necessario conoscere i tipi di dato gestiti da MySQL.

Il DBMS MySQL usa tre categorie fondamentali di tipi di dato:

1. numerici;
2. date e orari;
3. stringhe di caratteri.

Esistono principalmente nove tipi di dati numerici. Le convenzioni che si seguono nella tabella riassuntiva dei dati numerici successiva sono le seguenti:

- le indicazioni comprese fra parentesi quadre sono opzionali;
- M indica il numero totale di cifre rappresentate;
- D il numero di cifre decimali;
- tutti i tipi numerici escluso BIT possono avere le opzioni:
    - UNSIGNED - specifica che il numero è senza segno per cui non sono consentiti valori negativi;
    - ZEROFILL – indica di memorizzare i numeri con zeri davanti nel caso in cui la lunghezza sia inferiore a quella massima prevista.

Di seguito si riporta la tabella riassuntiva dei tipi di dato numerici.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tipo di dato	Descrizione
BIT [(M)]	È rappresentato con il numero di bit specificato da M (da 1 a 64; default 1)
TINYINT [(M)] [UNSIGNED] [ZEROFILL]	È rappresentato su 1 byte e può contenere 256 valori, da -128 a +127, oppure da 0 a 255 se UNSIGNED
SMALLINT [(M)] [UNSIGNED] [ZEROFILL]	È rappresentato su 2 byte e può contenere 65.536 valori, da -32.768 a +32.767, oppure da 0 a 65.535 se UNSIGNED
MEDIUMINT [(M)] [UNSIGNED] [ZEROFILL]	È rappresentato su 3 byte e può contenere 16.777.216 valori, da -8.388.608 a +8.388.607, oppure da 0 a 16.777.215 se UNSIGNED
INT [(M)] [UNSIGNED] [ZEROFILL]	È rappresentato su 4 byte e può contenere oltre 4 miliardi valori, da -2.147.483.648 a +2.147.483.647, oppure da 0 a 4.294.967.295 se UNSIGNED
BIGINT [(M)] [UNSIGNED] [ZEROFILL]	È rappresentato su 8 byte e può contenere circa 18 miliardi di miliardi di valori, da -9.223.372.036.854.775.808 a +9.223.372.036.854.775.807, oppure da 0 a 18.446.744.073.709.551.615 se UNSIGNED
FLOAT [(M,D)] [UNSIGNED] [ZEROFILL]	È rappresentato in virgola mobile a «precisione singola»; i suoi limiti teorici vanno da -3,402823466E+38 a -1,175494351E-38 e da 1,175494351E-38 a 3,402823466E+38, oltre allo zero
DOUBLE [(M,D)] [UNSIGNED] [ZEROFILL]	È rappresentato in virgola mobile a «precisione doppia»; i suoi limiti teorici vanno da -1,7976931348623157E+308 a -2,2250738585072014E-308 e da 2,2250738585072014E-308 a 1,7976931348623157E+308, oltre allo zero
DECIMAL [(M[,D])] [UNSIGNED] [ZEROFILL]	Rappresentano numeri «esatti», con M cifre Totali di cui D decimali. I valori di default sono 10 per M e 0 per D. I valore limite per questi dati sono gli stessi di DOUBLE. Il massimo di cifre consentite è 65 per M e 30 per D

Per i tipi DOUBLE e FLOAT i limiti reali dipendono dall'hardware e dal sistema operativo. Se M e D non sono indicati, i valori possono essere memorizzati fino ai limiti effettivi. Per questi dati, l'uso di UNSIGNED disabilita i valori negativi, ma non ha effetto sui valori massimi positivi memorizzabili.

La precisione di numeri in virgola mobile è affidabile fino alla settima cifra decimale (circa) per FLOAT e alla quindicesima (circa) per DOUBLE.

Esistono inoltre numerosi sinonimi per i dati numerici riportati nella tabella precedente. Segue la tabella dei sinonimi dei dati numerici.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tipo di dato	Sinonimo
BOOL BOOLEAN	TINYINT(1)
INTEGER	INT
DOUBLE PRECISION REAL	DOUBLE
DEC NUMERIC FIXED	DECIMAL

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Tipi di dato orario, di data e stringa

Per gli orari e le date, esistono principalmente cinque tipi di dato riassunti nella tabella successiva.

Tipo di dato	Descrizione
DATE	Rappresenta date comprese tra 1000-01-01 (1 gennaio 1000) e 9999-12-31 (31 dicembre 9999). MySQL Visualizza le date nel formato mostrato, ma consente di inserirle sotto forma di stringhe o numeri
DATETIME	Rappresenta una data e un'ora con lo stesso range visto per DATE. La visualizzazione è nel formato AAAA-MM-GG HH:MM:SS, In questo caso possono essere usati formati diversi per l'inserimento
TIMESTAMP [(M)]	Può essere usato per memorizzare i valori corrispondenti al timestamp Unix, corrispondente al numero di secondi trascorsi dalla mezzanotte del 1 gennaio 1970
TIME	Rappresenta un valore di orario (ore, minuti e secondi) che va da -838:59:59 a 838:59:59. Anche qui la visualizzazione avviene nel formato indicato , ma è possibile usare formati diversi per l'inserimento
YEAR [(2|4)]	Rappresenta, su quattro cifre, un anno compreso fra 1901 e 2155, oppure 0000. Su due cifre invece i valori vanno da 70 (1970) a 69 (2069).

I valori relativi al tempo possono essere inseriti sia come stringhe, sia come sequenze di numeri. MySQL consente di utilizzare, nel caso delle stringhe, diversi tipi di caratteri come separatori. L'importante, però, è che l'ordine dei valori sia sempre anno-mese-giorno-ore-minuti-secondi.

Quando si usano i separatori nelle stringhe è anche possibile omettere gli zeri non significativi. Per esempio, è consentito 2005-9-21, altrimenti è necessario usare 20050921.

L'operazione più comune lavorando con date orari è quella di estrarne i valori dal database e presentarli in un formato facilmente interpretabile dall'utente. Il formato della data e dell'orario può essere modificato attraverso la funzione DATE_FORMAT() o TIME_FORMAT() (quest'ultima solo per l'orario). La funzione DATE_FORMAT() riceve due argomenti: la data da formattare e una stringa composta da alcuni caratteri speciali preceduti dal simbolo «%» che indicano come tale data debba essere formattata.

In merito alla stringa di formattazione della funzione DATE_FORMAT(), è possibile riferirsi alla convenzione riportata nella tabella successiva.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Carattere	Descrizione
%d	giorno del mese numerico 01…31
%M	nome del mese January…December
%m	mese numerico 01…12
%H	ora 00…23
%i	minuti 00…59
%s	secondi 00…59
%Y	anno a quattro cifre
%y	anno a due cifre

Un esempio di utilizzo della stringa di formattazione con la funzione DATEFORMAT è il seguente: DATEFORMAT('2005-04-12 15:10:17','%d-%m-%Y %H:%i:%s') che permette di ottenere la formattazione: 12-04-2005 15:10:17.

La funzione inversa di DATE_FORMAT() è STR_TO_DATE() che riceve come argomenti la stringa contenente la data e la corrispondente stringa di formattazione della data di partenza, e restituisce un valore DATETIME, DATE o TIME a secondo del contenuto. Un esempio di utilizzo della funzione STR_TO_DATE() è il seguente: STR_TO_DATE('03/10/2009 12.33','%d.%m%Y %H.%i') che corrisponde alla seguente data 2009-10-03 12:33:00.

MySQL fornisce, inoltre, alcune funzioni di uso comune che ritornano valori di tipo data o orario e sono riportate nella tabella seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Funzione	Descrizione
NOW() CURRENT_TIMESTAMP() SYSDATE()	Forniscono la data e l'ora corrente nel formato AAAA-MM-GG HH:MM:SS o AAAAMMGGHHMMSS a seconda che si tratti di un contesto stringa o numerico
CURRDATE() CURRENT_DATE()	Forniscono la data corrente nel formato AAAA-MM-GG o AAAAMMGG a seconda che si tratti di un contesto stringa o numerico
CURRTIME() CURRENT_TIME()	Forniscono l'ora corrente nel formato HH:MM:SS o HHMMSS a seconda che si tratti di un contesto stringa o numerico
UNIX_TIMESTAMP()	Se non viene specificato alcun argomento fornisce lo Unix timestamp corrente, se invece si specifica un valore di tipo DATE, DATETIME, TIMESTAMP fornisce lo Unix timestamp corrispondente alla particolare data

Per la gestione di date e orari MySQL offre varie funzioni di utilità, tra cui quelle riportate nella tabella seguente.

Funzione	Descrizione
YEAR(), MONTH(), DAY(), WEEK()	Estraggono da un valore o campo data rispettivamente l'anno, il mese, il giorno, la settimana in formato numerico
HOUR(), MINUTE(), SECOND()	Estraggono da un valore orario rispettivamente l'ora, i minuti, i secondi in formato numerico
ADDDATE()	Aggiunge un intervallo temporale espresso in giorni a una data
SUBDATE()	Sottrae un intervallo temporale espresso in giorni a una data
DATEDIFF()	Effettua la differenza tra due date espressa in giorni

Si riportano, ora alcuni esempi di utilizzo di tali funzioni:

- SELECT ADDDATE('2020-07-19',5) produrrà il seguente risultato: 2020-07-24;
- SELECT SUBDATE('2020-07-19',5) produrrà il seguente risultato: 2020-07-14;
- SELECT ADDDATE('2020-07-19',-5) produrrà il seguente risultato: 2020-07-14;
- SELECT DATEDIFF(CURRDATE(),data_nascita)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

FROM Personale

WHERE matricola = '12345'

produrrà come risultato il numero di giorni che intercorrono tra la data odierna e la data di nascita del dipendente con matricola 12345.

I tipi di dato per le stringhe di caratteri offerti da MySQL sono diversi e sono riportati nella tabella seguente.

Tipo di dato	Descrizione
CHAR(M)	È una stringa di lunghezza fissa M completata con spazi a destra al momento della memorizzazione che sono eliminati in fase di lettura; la lunghezza prevista va da 0 a 255 caratteri
VARCHAR(M)	È una stringa lunghezza variabile con lunghezza massima dichiarabile di 65.535 caratteri
BINARY(M) VARBINARY(M)	Corrispondono a CHAR e VARCHAR, memorizza una stringa di byte invece che di caratteri; i valori BINARY sono completati a destra con valori zero
TINYTEXT TEXT[(M)] MEDIUMTEXT LONGTEXT	Memorizzano campi di testo: la lunghezza massima è 255 caratteri per TINYTEXT, 65.535 per TEXT, 16.777.215 per MEDIUMTEXT, 4.294.967.296 per LONGTEXT
ENUM('valore1', 'valore2',...)	Può contenere uno dei valori elencati nella definizione, oppure NULL o una stringa vuota, che viene assegnata quando si cerca di inserire un valore non valido; i valori possibili possono essere fino a 65.535
BLOB	Sono Binary Long OBjects (TINY, MEDIUM, LONG) gestiti come stringhe binarie (per esempio per memorizzare un'immagine)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Il comando CREATE

Il comando CREATE nel linguaggio DDL di SQL permette di creare database, tabelle, indici, vincoli di integrità su tabelle e tra tabelle, ecc.

Per la creazione di un database (prima, quindi, della creazione delle tabelle) il comando CREATE ha la seguente sintassi:

CREATE DATABASE <nome_database>;

Per poter utilizzare un database appena creato si utilizza la seguente sintassi:

USE <nome_database>;

Per la creazione di una tabella il comando CREATE nella sua forma base ha la seguente sintassi:

```
CREATE TABLE <nome_tabella> (
<nome_campo_1> <tipo_1> [<vincolo_1>],
…
<nome_campo_n> <tipo_n> [<vincolo_n>]);
```

Tra i vincoli che è possibile imporre, è necessario utilizzare:

- NOT NULL per indicare che il campo dovrà essere sempre avvalorato (NULL se è possibile non avvalorare il campo);
- la keyword DEFAULT che indica quale valore dovrà inserire il DBMS se tale campo all'atto dell'inserimento di una nuova riga non dovesse essere avvalorato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si fornisce ora un esempio di utilizzo di comando DDL per la creazione della tabella «Personale»:

```
CREATE TABLE Personale (
matricola VARCHAR(5) NOT NULL DEFAULT '',
id_dip VARCHAR(3) DEFAULT NULL,
nominativo VARCHAR(50) DEFAULT NULL,
data_nascita DATE NOT NULL,
qualifica VARCHAR(2) DEFAULT NULL,
stipendio DOUBLE DEFAULT 0
);
```

Il comando CREATE TABLE permette di definire contestualmente anche i vincoli di integrità referenziale relativi alla chiave primaria e alle chiavi esterne come riportate nell'esempio che segue:

```
CREATE TABLE Composizione (
id_prod VARCHAR(4),
id_comp VARCHAR(4),
unita_comp INTEGER,
CONSTRAINT ChiavePrimaria PRIMARY KEY(id_prod, id_comp),
CONSTRAINT ComponentiComposizione FOREIGN KEY(id_comp)
REFERENCES Componente(id_comp),
CONSTRAINT ProdottiComposizione FOREIGN KEY(id_prod)
REFERENCES Prodotto(id_prod),
);
```

Il comando CREATE, oltre alla creazione di database e tabelle, può servire anche a creare oggetti diversi tra i quali una tabella, una vista e un indice.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Una tabella (permanente) può essere creata a partire dal risultato di una query, come nell'esempio che segue:

```
CREATE TABLE <nome_tabella>
    SELECT <lista_campi>
    FROM <lista_tabelle>
    [WHERE <condizione>];
```

Una nuova vista può essere creata a partire dal risultato di una query. In SQL una vista è una tabella virtuale utilizzabile a tutti gli effetti come se fosse una tabella reale, ma che non fa parte della base di dati se non come query memorizzata. La sintassi della creazione della vista è:

```
CREATE VIEW <nome_vista> AS
    (SELECT <lista_campi>
    FROM <lista_tabelle>
    [WHERE <condizione>]);
```

Ad esempio, per creare la vista che, per ciascun dipartimento, fornisce il totale degli stipendi mensili, è possibile utilizzare il seguente comando:

```
CREATE VIEW Totale_Stipendi(id_dip,importo_totale_stipendi) AS
    (SELECT id_dip, SUM(stipendio)
    FROM Personale
    GROUP BY id_dip);
```

Oltre alle tabelle e alle viste, è possibile creare anche indici per velocizzare le ricerche su uno specifico campo di una tabella (soprattutto in caso di tabella con un numero elevato di tuple) come avviene per gli indici dei libri. La sintassi per la creazione dell'indice è:

```
CREATE INDEX <nome_indice>
        ON <nome_tabella>(<nome_campo>);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un esempio di utilizzo della creazione dell'indice potrebbe essere il seguente:

```
CREATE INDEX indice_nominativo
        ON Personale(nominativo);
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Formichi, F., Meini, G. (2018). Corso di Informatica (seconda edizione) Vol. 3., Zanichelli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).